

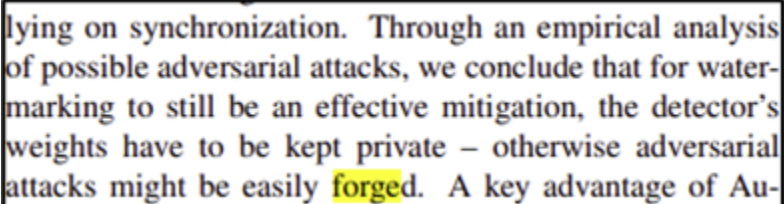


Introduction

This notebook is simply an individual extension of our group's `DeviousSeal`'s watermark removal attack on `AudioSeal`.

In our Milestone 3, we have explored and presented several watermark removal techniques. Previously, we found that the PGD attack was most effective in evading the watermark detector.

Knowing this, instead of removing the watermark, I'd like to use the same technique to forge the watermark. Being able to do this, this will allow malicious attackers to produce a malicious AI audio and apply the forgery on the audio to fool both detector and authorities, since they have no reason to suspect the audio is illegitimate, since it technically has a watermark.



lying on synchronization. Through an empirical analysis of possible adversarial attacks, we conclude that for watermarking to still be an effective mitigation, the detector's weights have to be kept private – otherwise adversarial attacks might be easily forged. A key advantage of Au-

Due to the effectiveness of the PGD attack in removing watermark and prompted by the fact that AudioSeal is hypothesis to be susceptible to forgery, I thought it would be possible to apply the same concept, but in the other direction. The above image is an excerpt from the `AudioSeal` paper, page 8.

Similarly, please install the below dependencies one by one to avoid runtime disconnection.

```
In [ ]: !pip install pesq --quiet
```

```
Preparing metadata (setup.py) ... done
Building wheel for pesq (setup.py) ... done
```

```
In [ ]: !pip install torchmetrics --quiet
```

```
890.6/890.6 kB 37.6 MB/s eta
0:00:00
```

```
In [ ]: !pip install audioseal --quiet
```

```

59.6/59.6 kB 4.9 MB/s eta
0:00:00
  Preparing metadata (setup.py) ... done
117.0/117.0 kB 10.3 MB/s et
a 0:00:00
  Preparing metadata (setup.py) ... done
79.5/79.5 kB 7.0 MB/s eta 0:0
0:00
  Building wheel for julius (setup.py) ... done
  Building wheel for antlr4-python3-runtime (setup.py) ... done

```

Before we start:-

Using the same audio file from Milestone 3

For consistency, I'll be using the same audio file from our groups Milestone 3.

```

In [ ]: # all imports used
import os
import time
import torch
import torchaudio
import torch.nn as nn
import torch.optim as optim
import IPython.display as ipd
import matplotlib.pyplot as plt
from audioseal import AudioSeal
from torchmetrics.audio.pesq import PerceptualEvaluationSpeechQuality

```

```

In [ ]: # retrieving dataset
os.makedirs('LibriSpeech_dataset', exist_ok=True)

librispeech_test = torchaudio.datasets.LIBRISPEECH(root='LibriSpeech_data

# the same audio sample from M3, which is the first audio from this datas
demo_audio = [librispeech_test[0]]
# extracting just the audio (this is not watermarked at all, straight fro
demo_audio = demo_audio[0][0].unsqueeze(0)

```

```
100%|██████████| 331M/331M [00:25<00:00, 13.6MB/s]
```

Ensure collab is running on GPU

```

In [ ]: device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"currently using device: {device}")
demo_audio = demo_audio.to(device)
sample_rate = 16000

```

currently using device: cuda

The below code is tweaked a bit from our PGD attack for watermark removal. However, the main difference is the penalty we are trying to minimize. Back in Milestone 3, we were trying to minimize the detection probability. Here however, we are trying to increase the probability that a non-watermarked audio is mistaken as watermarked audio. Previously, we wanted to minimize the detection probability.

Currently, we want to increase the detection probability. But I need to change the target in the PGD attack. Instead, the loss I am trying to minimize here is the difference between the watermark detection probability and the threshold.

In other words, the attack's objective is to raise the detection probability of non-watermarked audio so that it approaches or exceeds a defined threshold. The penalty is applied only when the detection probability is below the threshold, and the goal is to reduce this penalty, thereby increasing the overall detection probability. Since 50% is the minimum threshold of what is considered watermarked we had set it to that.

```
penalty = torch.relu(self.threshold - class_1_probs)
```

```
In [ ]: class WatermarkDetectorWrapper():
    def __init__(self, model, threshold):
        self.model = model
        self.threshold = threshold
        self.calculate_detection_accuracy = self.detection_accuracy_audio
        self.get_loss = self.loss_audioseal

    def loss_audioseal(self, audio):
        # perform forward pass through the model
        results, _ = self.model(audio, sample_rate=sample_rate)

        # loss calculation for watermark probability
        watermarked_probs = results[:, 1, :]
        penalty = torch.relu(self.threshold - watermarked_probs)
        avg_penalty = torch.mean(penalty)

        return avg_penalty

    def detection_accuracy_audioseal(self, audio):
        # forward pass through the detector
        result, _ = self.model.detect_watermark(audio, sample_rate=sample_rate)
        return result
```

```
In [ ]: def pgd_forge_attack(detector, audio, steps, alpha, eps):
    start_time = time.time()
    perturbation = torch.zeros_like(audio, requires_grad=True)
```

```

# set up optimizer
optimizer = optim.Adam([perturbation], lr = alpha)

for i in range(steps):
    optimizer.zero_grad()

    # apply perturbation and calculate the loss
    perturbed_audio = audio + perturbation
    loss = detector.get_loss(perturbed_audio)

    loss.backward()

    optimizer.step()

    # ensure perturbation stays within bounds
    perturbation.data = torch.clamp(perturbation.data, -eps, eps)

    # get accuracy score
    detection_accuracy = detector.calculate_detection_accuracy(pertur

    # calculate the signal-to-noise ratio (snr) between the original
    snr = 10 * torch.log10(torch.mean(perturbed_audio**2) / torch.me

    # this attack requiried
    if i % 100 == 0:
        print(f"Iteration {i}: Loss = {loss.item():.3f}, Detection accu

    # stoppe when audio is detetcred as 100% accurate
    if detection_accuracy == 1:
        break

print(f"Attack finished in {time.time() - start_time:.2f} seconds")
return audio + perturbation

```

```

In [ ]: # loading the AudioSeal watermark detector model
detector_model = AudioSeal.load_detector("audioseal_detector_16bits").to(

# creating an instance of the WatermarkDetectorWrapper with a threshold o
# if im able to increase it to above 50%, it would be consider a win
detector = WatermarkDetectorWrapper(model=detector_model, threshold=0.5)

```

```

Downloading: "https://huggingface.co/facebook/audioseal/resolve/main/dete
ctor_base.pth" to /root/.cache/audioseal/94c8df0b1d5ea8e45af4c884
100%|██████████| 33.1M/33.1M [00:00<00:00, 42.0MB/s]

```

Similarly, the parameters plays a big part in balacning the audio quality and the effectiveness of the attack. For example, a lower learning rate (alpha) will cause the audio to be classified watermarked in earlier iterations, but the SNR / quality of the audio can go under 35. Which is still considered

acceptable, but since I can find a better solution, I had settled with the following parameters after some exploration.

After some trial and error I found that after 100 steps it was able to significantly increase the watermark probability. After some fine tuning, I decided to increase the steps/iteration to 2000 steps just for completeness, but it should end between 800 to 900 steps/iteration .

```
In [ ]: # starting the attack
print("Starting the whitebox attack...")
forged_audio = pgd_forge_attack(detector, demo_audio, steps = 2000, alpha
```

```
Starting the whitebox attack...
Iteration 0: Loss = 0.462, Detection accuracy = 1.14%, SNR = 73.38
Iteration 100: Loss = 0.066, Detection accuracy = 80.71%, SNR = 47.17
Iteration 200: Loss = 0.019, Detection accuracy = 93.51%, SNR = 45.56
Iteration 300: Loss = 0.007, Detection accuracy = 98.24%, SNR = 45.18
Iteration 400: Loss = 0.006, Detection accuracy = 98.55%, SNR = 45.11
Iteration 500: Loss = 0.005, Detection accuracy = 98.84%, SNR = 45.04
Iteration 600: Loss = 0.004, Detection accuracy = 98.94%, SNR = 44.99
Iteration 700: Loss = 0.004, Detection accuracy = 98.92%, SNR = 44.94
Iteration 800: Loss = 0.002, Detection accuracy = 99.62%, SNR = 44.84
Iteration 900: Loss = 0.002, Detection accuracy = 99.57%, SNR = 44.83
Iteration 1000: Loss = 0.001, Detection accuracy = 99.62%, SNR = 44.80
Iteration 1100: Loss = 0.001, Detection accuracy = 99.71%, SNR = 44.78
Iteration 1200: Loss = 0.005, Detection accuracy = 94.51%, SNR = 44.59
Iteration 1300: Loss = 0.001, Detection accuracy = 99.82%, SNR = 44.05
Iteration 1400: Loss = 0.000, Detection accuracy = 99.88%, SNR = 44.04
Iteration 1500: Loss = 0.000, Detection accuracy = 99.92%, SNR = 44.03
Attack finished in 195.43 seconds
```

An SNR above 40 is actually pretty good, as it means that the ratio of signal to noise is not too large.

```
In [ ]: #printing results
print('Detection Probability of Demo Audio')
result1, _ = detector_model.detect_watermark(demo_audio, sample_rate=samp
print(f"Detection Result(did not go tru AudioSeal WM generator aka ◇ wate

print('\nDetection Probability of Forged Demo Audio')
result2, _ = detector_model.detect_watermark(forged_audio, sample_rate= s
print(f"Detection Result(Forged watermark ◇◇): {result2*100:.2f}%")
```

```
Detection Probability of Demo Audio
Detection Result(did not go tru AudioSeal WM generator aka ◇ watermark):
1.14%
```

```
Detection Probability of Forged Demo Audio
Detection Result(Forged watermark ◇◇): 100.00%
```

As we can see above, the initial `demo_audio` that had not gone through the AudioSeal watermark generator is very very low, which is expected.

However, by exploiting the AudioSeal model, I was able to find a perturbation via `PGD` to pretend that the non-watermarked audio is actually watermarked. Promptly fooling the detector model.

```
In [ ]: # code to plot spectrogram & waveform
def plot_waveform_and_spectrogram(waveform, sample_rate, title):
    """
    plots both the waveform and spectrogram of an audio sample.

    parameters:
    waveform (tensor): the audio data (waveform) to be visualized.
    sample_rate (int): the sample rate of the audio.
    title (str): the title for the plot.

    this function creates a side-by-side view of the waveform and its spectrogram
    """
    # squeeze removes any unnecessary dimensions, detach for no gradients
    waveform = waveform.squeeze().detach().cpu().numpy()

    # determine the number of frames and create a time axis based on the
    num_frames = waveform.shape[-1]
    time_axis = torch.arange(0, num_frames) / sample_rate

    # create a figure with two subplots, one for the waveform and one for the spectrogram
    figure, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 4))

    # plot the waveform on the first subplot
    ax1.plot(time_axis, waveform, linewidth=1)
    ax1.grid(True)

    # plot the spectrogram on the second subplot
    ax2.spectrogram(waveform, Fs=sample_rate)

    # set the overall title and show the plots
    figure.suptitle(f"{title} - waveform and spectrogram")
    plt.show()

def play_audio(waveform, sample_rate):
    """
    plays the audio directly in a notebook cell.

    parameters:
    waveform (tensor): the audio data (waveform) to be played.
    sample_rate (int): the sample rate of the audio.

    this function handles both mono (1 channel) and stereo (2 channel) audio
    """
    # squeeze any extra dimensions if needed
    if waveform.dim() > 2:
```

```

        waveform = waveform.squeeze(0)
        waveform = waveform.detach().cpu().numpy()

        # check how many channels the audio has
        num_channels, *_ = waveform.shape
        # if mono audio, play normally
        if num_channels == 1:
            ipd.display(ipd.Audio(waveform[0], rate=sample_rate))
        # if stereo audio, play both channels
        elif num_channels == 2:
            ipd.display(ipd.Audio((waveform[0], waveform[1]), rate=sample_rate))
        # raise an error if there are more than 2 channels, since they're not
        else:
            raise ValueError("waveform with more than 2 channels is not supported")

```

```

In [ ]: # initializing pesq
        pesq = PerceptualEvaluationSpeechQuality(fs=16000, mode='wb')

        # calculate pesq score for the original audio and forged watermarked audio
        pesq_score_demo = pesq(forged_audio.squeeze(), demo_audio.squeeze())

        # PESQ score between demo audio & the forged audio
        print(f"Forgery Attack--> PESQ Score: {pesq_score_demo:.2f}")

```

Forgery Attack--> PESQ Score: 4.48

Interestingly enough, looking back between the PGD attack of forging and removing watermark (in Milestone 3), forging has the highest quality of audio after an attack in comparison to the others. This indicates that the audio when listened by human, is very very much likely hard to notice any difference at all!

4.5 is the maximum PESQ score, for reference.

```

In [ ]: print('+----Original Audio----+')
        play_audio(demo_audio,sample_rate)

        print('\n+----Forged Audio----+')
        play_audio(forged_audio,sample_rate)

```

+----Original Audio----+

Your browser does not support the audio element.

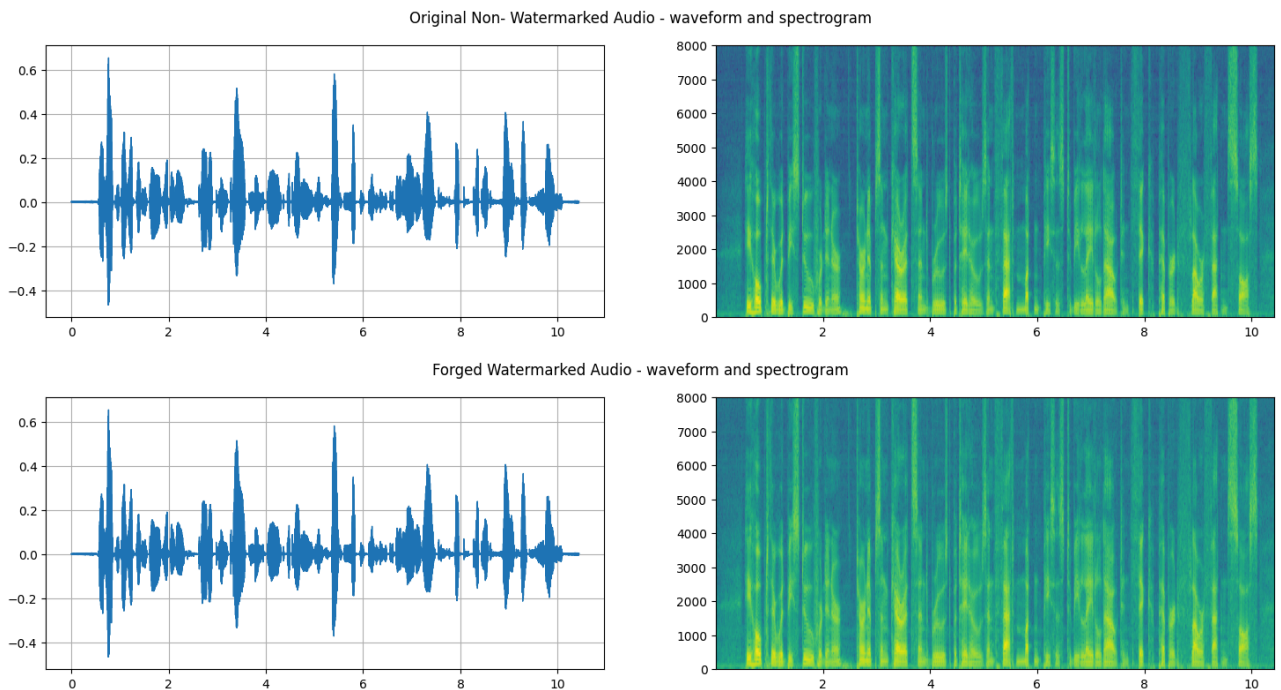
+----Forged Audio----+

Your browser does not support the audio element.

```

In [ ]: plot_waveform_and_spectrogram(demo_audio,sample_rate,'Original Non- Watermark')
        plot_waveform_and_spectrogram(forged_audio,sample_rate,'Forged Watermarked')

```



```
In [ ]: result_prob_dict = {'original(watermarked)': (1.0, 4.504801273345947),
                           'query': (0.4988440275192261, 1.91431040763855) ,
                           'pso': (0.45358768105506897, 1.754843521118164),
                           'pgd(removal)': (0.2969813048839569, 3.8860957622528076),
                           # the above results are from M3 so hardcoded (but it shou
                           'pgd(forgery)': (result2,pesq_score_demo.item())

                           }
```

```
result_prob_dict
```

```
Out[ ]: {'original(watermarked)': (1.0, 4.504801273345947),
         'query': (0.4988440275192261, 1.91431040763855),
         'pso': (0.45358768105506897, 1.754843521118164),
         'pgd(removal)': (0.2969813048839569, 3.8860957622528076),
         'pgd(forgery)': (1.0, 4.482587814331055)}
```

```
In [ ]: # gathering values from above dict to plot
attack_types = list(result_prob_dict.keys())
probabilities = [value[0] for value in result_prob_dict.values()]
pesq_scores = [value[1] for value in result_prob_dict.values()]

# plotting results
bar_width = 0.35
index = range(len(attack_types))
fig, ax = plt.subplots(figsize=(10, 6))

# PESQ scores bars
bar1 = ax.bar([i - bar_width / 2 for i in index], pesq_scores, bar_width,

# detection probabilities bars
bar2 = ax.bar([i + bar_width / 2 for i in index], probabilities, bar_widt
```

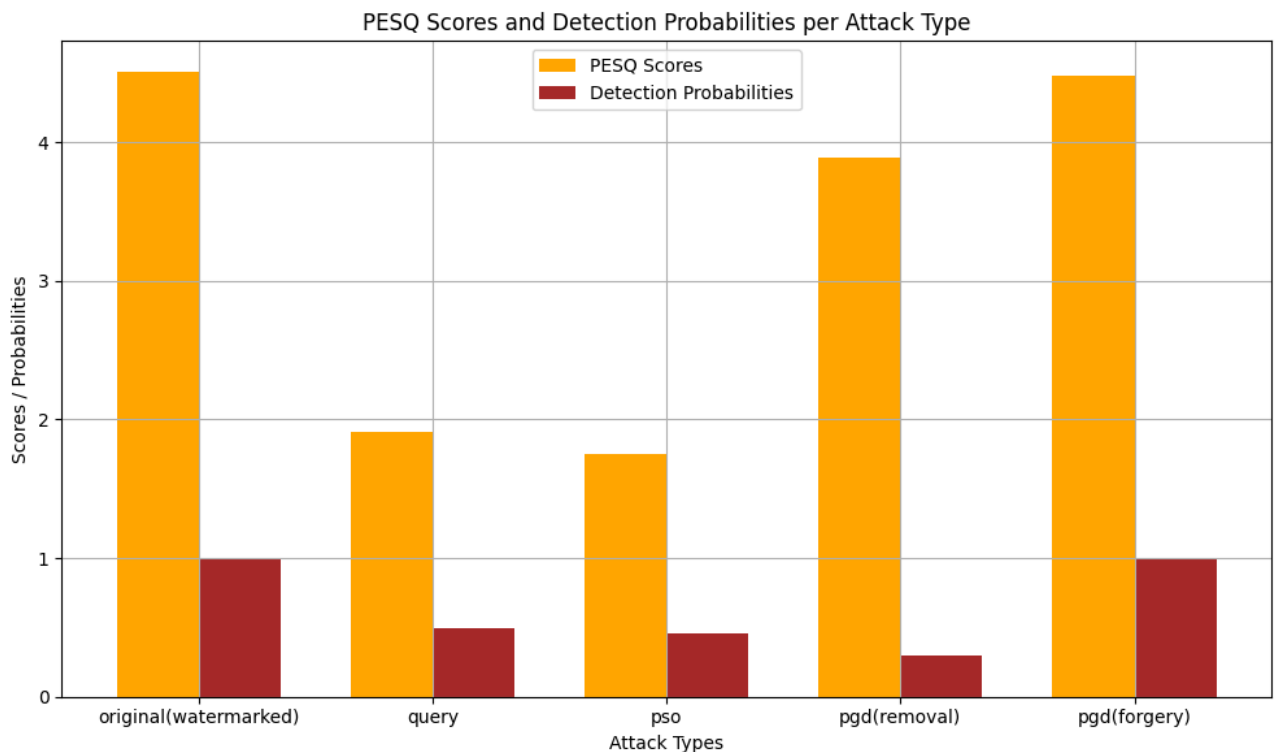
```

ax.set_xlabel('Attack Types')
ax.set_ylabel('Scores / Probabilities')
ax.set_title('PESQ Scores and Detection Probabilities per Attack Type')

ax.set_xticks(index)
ax.set_xticklabels(attack_types)
ax.legend()
ax.grid(True)

plt.tight_layout()
plt.show()

```



In conclusion, the attacks that have been presented in Milestone 3 & 4 are simply the tip of the iceberg, and there is actually much more sophisticated ways in attacking AudioSeal, that we may have missed due to our own constraints. As a group, we did not expect forgery to be the most effective attack due to our own bias thinking of assuming removing something might be easier than recreating something.

The effectiveness of our attacks, primarily whitebox, are potentially due to the fact we have full knowledge of AudioSeals model so we can 'cheat' it somehow. However, as mentioned, realistically speaking, if we could have

implemeted a blackbox attack that can be as superior as a whitebox attack, that would certainly be an incredible attack, since most of the time, attacker wont have access to the model. My teammate has alos explored another type of attack, that is interestingly enough, quite practical and could be ***almost*** asgood enough as a white box attack.

Till then, thank you for your time & attention in our attack against
AudioSeal .

-mm-

In []: